

VERİLERİN GÖSTERİMİ

[Kaynak](#)

İÇERİK

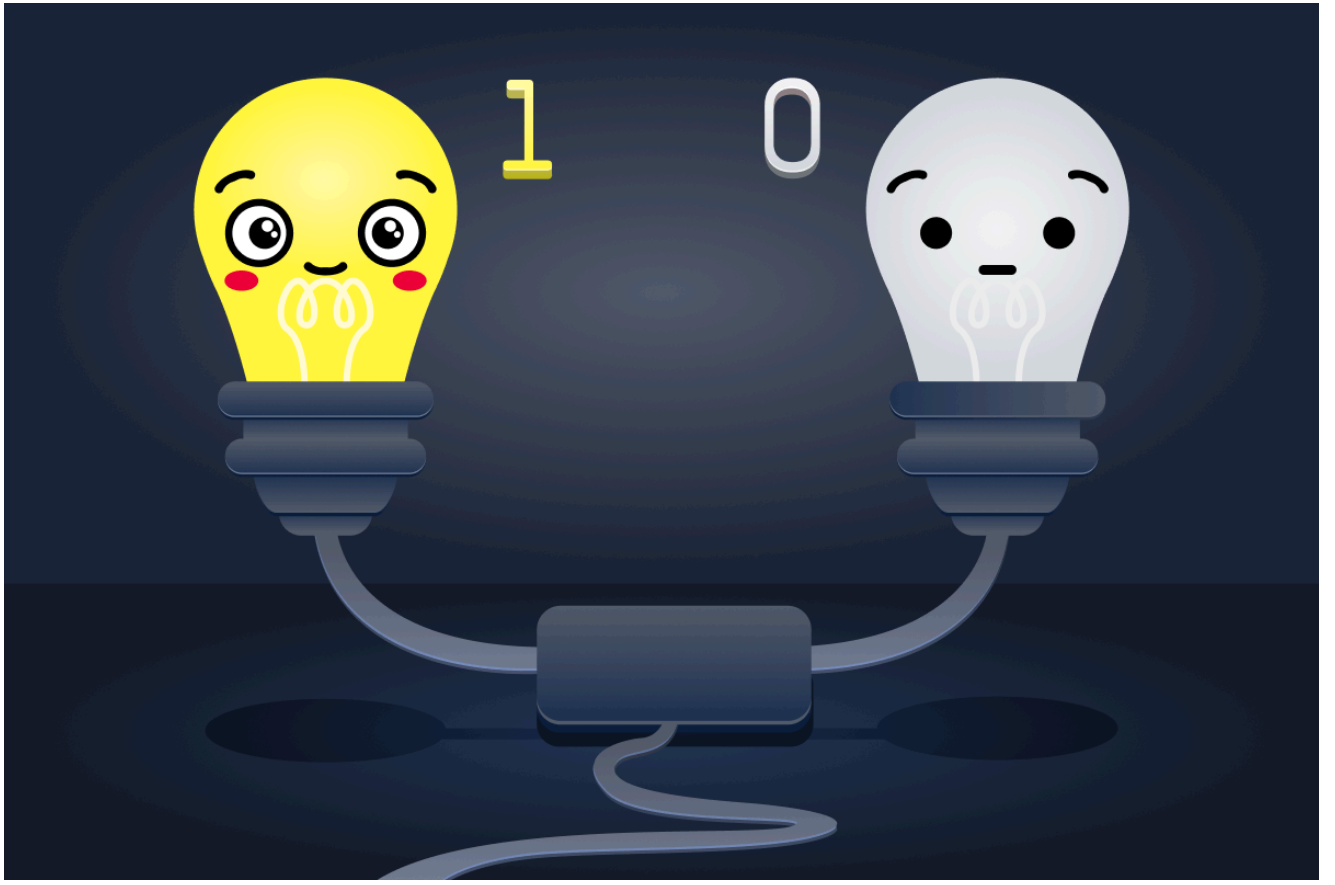
1. Data Representation: Decimal vs. Binary Sistemler
2. Representing Colors (Renkleri Gösterme)
3. Sayılar: Ondalık Sistemden Onaltılık Sisteme
4. Sayı Sistemleri ve Renk Temsili: Nihai Sistem Mimarisi Özeti

Data Representation: Decimal vs. Binary Sistemler

Bu aşamada, insan algısı ile bilgisayar donanımı arasındaki temel veri işleme ve temsil etme farklılıklarını analiz ediyoruz.

İnsanlar olarak günlük hayattaki ölçüm, sayım ve hesaplama işlemlerimizde doğal bir refleks olarak **Decimal (Onluk)** sayı sistemini (0'dan 9'a kadar olan rakamlar) kullanırız. Ancak bilgisayar mimarisi, fiziksel donanım seviyesinde yalnızca iki elektriksel durumu okuyup yazabilecek şekilde tasarlanmıştır: Elektrik var (1) veya elektrik yok (0). Bu sistem **Binary (İkili)** sayı sistemi olarak adlandırılır.

ÖNEMLİ: Bilgisayar ekranında gördüğümüz spesifik bir rengin (örneğin mor) render edilmesi veya sistemin gerçekleştirdiği en karmaşık matematiksel hesaplamalar, arka planda sadece hafızadaki bu iki **state (durum)** üzerinden işlenir. Temel amaç, onluk sistemdeki algımızın ve görsel verilerin, donanımın anlayabileceği bu 1 ve 0'lara nasıl çevrildiğinin mantığını kavramaktır.



Representing Colors (Renkleri Gösterme)

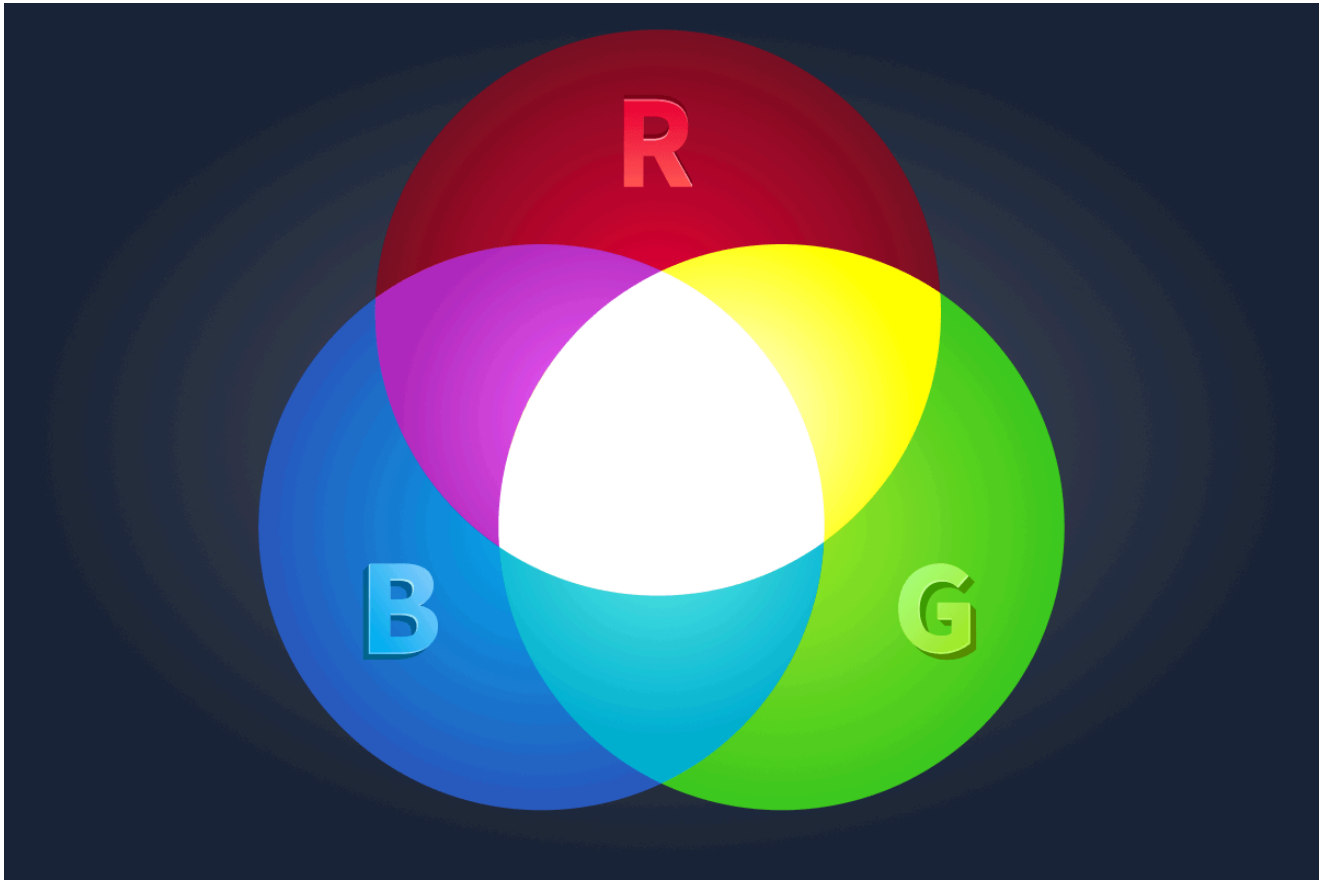
Temel RGB Mantığı ve 3-Bit Renk Uzayı

Bilgisayar sistemlerinde renklerin temsili, fiziksel ışık spektrumunun dijital ortama aktarılması mantığına dayanır. Bu sistemin temelinde **RGB (Red, Green, Blue)** adı verilen üç ana renk kanalı bulunur. Bu kanalları, yoğunluğu ayarlanabilen üç farklı ışık kaynağı veya potansiyometre (düğme) gibi düşünebiliriz. Bu üç temel rengin farklı oranlarda karıştırılmasıyla teorik olarak istenilen her renk elde edilebilir.

Konuyu en temel yapı taşına indirgeyecek olursak; her bir renk kanalının (kırmızı, yeşil, mavi) yalnızca iki durumu (state) olduğunu varsayalım: **Açık (1)** veya **Kapalı (0)**.

Bir bilgisayarın bu temel mimaride sahip olabileceği renk kombinasyonlarını hesaplamak için basit bir olasılık matematiği kullanırız:

$2 \text{ (Kırmızı Durumu)} \times 2 \text{ (Yeşil Durumu)} \times 2 \text{ (Mavi Durumu)} = 8 \text{ farklı renk.}$



Bilgisayar sistemleri, bu 8 rengi hafızada tutabilmek ve işleyebilmek için sadece hangi rengin aktif (1), hangisinin pasif (0) olduğunu bilmeye ihtiyaç duyar. İkili sistemdeki bu 1 ve 0 değerlerinin her birine **bit (Binary Digit - dijital verinin en küçük ölçü birimi)** adı verilir. Dolayısıyla, bu kısıtlı renk uzayını ifade etmek için **3-bit** uzunluğunda bir veri bloğu yeterlidir.

Aşağıdaki tablo, bu 3-bitlik sistemde renklerin binary (ikili) karşılıklarını ve elde edilen sonuçları göstermektedir:

Binary (İkili) Gösterim	Arka Plandaki Mantık (RGB Durumu)	Elde Edilen Renk
000	Tüm kanallar kapalı (ışık yok).	Siyah (Black)
001	Sadece Mavi kanalı açık.	Mavi (Blue)
010	Sadece Yeşil kanalı açık.	Yeşil (Green)
100	Sadece Kırmızı kanalı açık.	Kırmızı (Red)
011	Yeşil ve Mavi kanalları açık.	Camgöbeği (Cyan)
101	Kırmızı ve Mavi kanalları açık.	Eflatun (Magenta)
110	Kırmızı ve Yeşil kanalları açık.	Sarı (Yellow)
111	Tüm kanallar açık (maksimum ışık karışımı).	Beyaz (White)

24-Bit Renk Derinliğine (True Color) Geçiş

Gerçek dünya senaryolarında ve grafik tasarım gibi detay gerektiren alanlarda 8 renkli bir palet son derece yetersizdir. İnsan gözünün algılayabildiği milyonlarca renk tonunu dijital ortamda kusursuz bir şekilde simüle edebilmek için, her bir kanalın sadece "açık" veya "kapalı" olmasının ötesine geçerek, o kanalın **ışık şiddetini (intensity)** hassas bir şekilde ayarlayabilmemiz gerekir.

Bunu başarmak için, her bir renk kanalına (R, G ve B) 1 bit yerine **8 bit** tahsis edilir. Bilgisayar terminolojisinde 8 bitlik bu veri bloklarına **byte** veya **octet** denir.

8 bitlik bir alanın alabileceği maksimum değer olasılığı şudur:

$2^8 = 256$ farklı seviye (0 dahil olmak üzere 0'dan 255'e kadar).

Eğer elimizdeki üç kanalın her biri 256 farklı yoğunluk seviyesine sahip olursa, elde edebileceğimiz toplam renk sayısı olağanüstü bir boyuta ulaşır:

$256 \text{ (Kırmızı)} \times 256 \text{ (Yeşil)} \times 256 \text{ (Mavi)} = \mathbf{16.777.216 \text{ renk.}}$

ÖNEMLİ: Bu hesaplama sonucunda anlıyoruz ki, günümüzde standart olarak kullanılan bir rengi hafızada temsil edebilmek için **3 x 8 bit**, yani toplamda **24 bit (3 byte)** veri alanına ihtiyacımız vardır. Bu standart sektörde "True Color" (Gerçek Renk) olarak da bilinir.

Hexadecimal (Onaltılık) Gösterim ve Veri Optimizasyonu

24 bitlik bir renk verisini doğrudan binary olarak yazmak son derece verimsizdir ve okunabilirliği sıfıra indirir. Örneğin, belirli bir yeşil tonunun hafızadaki karşılığı olan 24 bitlik raw (ham) veri şöyledir:

```
10100011 11101010 00101010
```

Yukarıdaki çıktı, sırasıyla Kırmızı, Yeşil ve Mavi kanallarının yoğunluklarını belirten 3 ayrı byte'ı (toplam 24 bit) temsil eder. Bu formatı konfigürasyon dosyalarında, CSS kodlarında veya terminal üzerinde okumak/yazmak hata yapmaya çok müsaittir.

Bu karmaşayı çözmek için **Hexadecimal (Onaltılık - Base16)** sayı sistemi devreye girer. Hexadecimal sistemin bilgisayar bilimlerindeki en büyük gücü, **tam olarak 4 bitlik (bir nibble) binary veriyi tek bir karakterle kusursuzca temsil edebilmesidir.**

Aşağıdaki tablo, 4 bitlik binary blokların nasıl tek bir Hexadecimal karaktere dönüştüğünü gösterir:

Hex Karakteri	Binary Karşılığı (4-bit)		Hex Karakteri	Binary Karşılığı (4-bit)
0	0000		8	1000
1	0001		9	1001
2	0010		A	1010

Hex Karakteri	Binary Karşılığı (4-bit)		Hex Karakteri	Binary Karşılığı (4-bit)
3	0011		B	1011
4	0100		C	1100
5	0101		D	1101
6	0110		E	1110
7	0111		F	1111

Bu mantığı yukarıdaki karmaşık 24 bitlik verimize uyguladığımızda, her bir byte'ı (8 bit) tam olarak **iki Hexadecimal karakter** ile özetleyebiliriz.

Dönüşüm Adımları:

1. **Kırmızı Kanalı (1. Byte):** 10100011

- 1010 -> **A**
- 0011 -> **3**
- Hex Sonucu: **A3**

2. **Yeşil Kanalı (2. Byte):** 11101010

- 1110 -> **E**
- 1010 -> **A**
- Hex Sonucu: **EA**

3. **Mavi Kanalı (3. Byte):** 00101010

- 0010 -> **2**
- 1010 -> **A**
- Hex Sonucu: **2A**

Nihai Sonuç: 10100011 11101010 00101010 binary verisi, grafik programlarında ve web geliştirmede doğrudan **A3EA2A** olarak yazılır. Bu dönüşüm sayesinde 24 karakterlik okunması zor bir veri bloğu, 6 karakterlik kompakt, akılda kalıcı ve insan tarafından kolayca işlenebilir bir standarda indirgenmiş olur.

DİKKAT (Kritik Teknik Çıkarımlar):

- Gerçek dünya uygulamalarında (Real-life applications) renkler her zaman **24 bit (3 byte)** olarak temsil edilir.
- Bu 3 byte'ın her biri kendi içerisinde **256 farklı değer** (intensity/yoğunluk) alabilir.
- Binary'den Hexadecimal'e geçişte her **4 bit**, tek bir Hexadecimal haneye (digit) denk gelir.
- Bunun doğal bir sonucu olarak, her **1 byte (8 bit)** her zaman **2 Hexadecimal hane** ile ifade edilir (Örn: FF = 255).

Sayılar: Ondalık Sistemden Onaltılık Sisteme

Fiziksel Katman ve Sayı Sistemlerinin Temeli

İnsanlar evrimsel süreçleri gereği gündelik hayatta 10 parmaklı bir anatomiye sahip oldukları için **Decimal (Onluk - Base-10)** sayı sistemini kullanırlar. Ancak bilgisayar mimarisi biyolojik değil, elektronik ve fiziksel prensiplere dayanır. Dijital sistemlerin "anlayabildiği" yegane dil, fiziksel donanım üzerindeki iki farklı durumdan ibarettir.

Bu iki temel durum (0 ve 1), sistemin mimarisine göre farklı fiziksel gerçeklikleri temsil eder:

- **Transistör-Transistör Mantığı (Transistor-Transistor Logic):** Elektriksel voltaj seviyelerini kullanır. **Low (Düşük)** voltaj (örn. 0-0.8V arası) 0'ı, **High (Yüksek)** voltaj (örn. 2-5V arası) 1'i temsil eder. Elektrik akımı ya geçer ya da bloklanır.
- **Sabit Disk Sürücüler (Hard Disk Drives):** Manyetik polarizasyonu kullanır. Kuzey (North) ve Güney (South) kutupları 0 ve 1 değerlerini tutar.
- **Fiber Optik Sistemler (Fiber Optics):** Işığın varlığı (1) veya yokluğu (0) esasına dayanır.

Biz siber güvenlik veya sistem uzmanları olarak fiziksel katmandaki bu voltaj veya ışık farklılıklarıyla doğrudan ilgilenmeyiz; asıl odak noktamız, bu iki dijital verinin (0 ve 1) karmaşık verileri ve sayıları nasıl inşa ettiğidir.

Decimal (Onluk) Sistemin Matematiksel Anatomisi

Binary sisteme geçmeden önce, zihnimizin otomatik olarak işlediği Decimal (Base-10) sistemin arka planındaki matematiği parçalamamız gerekir. Onluk sistemde sağdan sola doğru her basamak, 10'un artan bir kuvveti (power) ile çarpılır.

Örneğin, gündelik hayatta kullandığımız 213 sayısının matematiksel olarak çözümlenmiş hali aşağıdaki gibidir:

$$213 = (2 \times 100) + (1 \times 10) + (3 \times 1)$$

Yukarıdaki basit hesaplama, aslında basamak değerlerinin arka plandaki ağırlıklarını gösterir. Teknik olarak her basamak 10'un kuvvetleridir.

Bunu tam matematiksel notasyona (üs mantığına) döktüğümüzde sistemin temel formülünü elde ederiz:

$$213 = (2 \times 10^2) + (1 \times 10^1) + (3 \times 10^0)$$

Matematik kuralları gereği $10^2 = 100$, $10^1 = 10$ ve her sayının sıfırıncı kuvveti 1 olduğundan $10^0 = 1$ 'dir. Bu mantık, tüm sayı sistemlerinin çevrilmesinde kullanacağımız evrensel temeldir.

Binary (İkili - Base-2) Sistem ve Matematiksel Çözümleme

Binary (Base-2) sistemin çalışma prensibi, Decimal sistemle tamamen aynıdır. Tek kritik fark; taban değerinin 10 yerine 2 olması ve kullanabileceğimiz rakam havuzunun sadece **0** ve **1** ile sınırlı olmasıdır. Bu sistemde sağdan sola her basamak, 2'nin artan bir kuvveti ile çarpılır.

Örneğin, binary sistemdeki **1001** verisini ele alalım ve Decimal sisteme dönüştürelim:

$$\begin{aligned}1001 &= (1 \times 2^3) + (0 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) \\1001 &= (1 \times 8) + (0 \times 4) + (0 \times 2) + (1 \times 1) \\1001 &= 8 + 0 + 0 + 1 = 9\end{aligned}$$

Yukarıdaki çıktıdan anladığımız üzere, binary formatındaki 1001 verisinin bizim anlayacağımız dildeki tam karşılığı 9'dur. Çarpan sıfır olduğunda, o basamağın değeri toplam sonuca etki etmez.

Bu mantığı iyice oturtmak için 4-bitlik (bir nibble) diğer temel binary dizilimlerin çevrimlerine bakalım:

$$\begin{aligned}0000 &= 0 + 0 + 0 + 0 = 0 \\0001 &= 0 + 0 + 0 + 1 = 1 \\0010 &= 0 + 0 + 2 + 0 = 2 \\0011 &= 0 + 0 + 2 + 1 = 3\end{aligned}$$

Dikkat ederseniz, aktif olan bit (1 olan hane) sadece kendi basamak ağırlığını sonuca eklemektedir. En sağdaki bit 2^0 (1), bir solundaki 2^1 (2) ağırlığındadır.

Eğer 4-bitlik alanın alabileceği en yüksek değerleri hesaplamak istersek:

$$\begin{aligned}1100 &= 8 + 4 + 0 + 0 = 12 \\1101 &= 8 + 4 + 0 + 1 = 13 \\1110 &= 8 + 4 + 2 + 0 = 14 \\1111 &= 8 + 4 + 2 + 1 = 15\end{aligned}$$

Bu çıktı bize çok kritik bir bilgi veriyor: 4-bitlik bir hafıza bloğunun (nibble) alabileceği en yüksek Decimal değer 15'tir (0 dahil toplam 16 farklı durum).

Hexadecimal (Onaltılık - Base-16) Sistem

Önceki adımlarda 4-bitlik bir binary dizilimin (Örn: 1111) onluk sistemde 15'e denk geldiğini gördük. **Hexadecimal (Base-16)** sistem, bilgisayar bilimlerinde 4-bitlik binary gruplarını **tek bir karakter** ile ifade etmek için yaratılmış mükemmel bir optimizasyon standardıdır.

Bu sistemde rakamlar 0'dan başlar ancak Decimal sistemdeki 10, 11, 12, 13, 14 ve 15 sayıları tek bir hanede ifade edilemeyeceği için alfabadeki harfler (**A, B, C, D, E, F**) ile temsil edilir.

Decimal (Onluk)	Hexadecimal (Onaltılık)	Binary (İkili)
0 - 9	0 - 9	0000 - 1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

DİKKAT: Bir hafıza dökümünü (memory dump) veya ağ paketini analiz ederken gördüğünüz her bir Hex karakteri, arka planda kesinlikle **4 bitlik** bir binary dizilimini temsil eder.

Hexadecimal'den Decimal'e Çevrim Mantığı:

Decimal ve Binary çevrimlerindeki evrensel kural burada da geçerlidir; sadece tabanımız 16'dır. Örneğin, bellekteki **9BDF** hex değerinin onluk sistemdeki karşılığını bulalım:

$$\begin{aligned} 9BDF &= (9 \times 16^3) + (11 [B] \times 16^2) + (13 [D] \times 16^1) + (15 [F] \times 16^0) \\ 9BDF &= (9 \times 4096) + (11 \times 256) + (13 \times 16) + (15 \times 1) \\ 9BDF &= 36864 + 2816 + 208 + 15 = 39,903 \end{aligned}$$

Yukarıdaki çıktı, harflerin çevrim sırasında kendi Decimal karşılıklarına (B=11, D=13, F=15) dönüştürülerek 16'nın kuvvetleriyle çarpıldığını açıkça göstermektedir.

Octal (Sekizlik - Base-8) Sistem [Alternatif Temsil]

Hexadecimal sistem veriyi 4-bitlik bloklar halinde (Base-16) gruplarken, **Octal (Base-8)** sistem veriyi **3-bitlik** bloklar halinde gruplar. Kullanılabilir rakam havuzu **0 ile 7** arasındadır. (Özellikle Linux dosya izinlerinde (File Permissions - rwxr-xr-x) sıkça karşılaştığımız `chmod 777` veya `chmod 644` komutları doğrudan Octal sistemi kullanır).

Decimal	Octal	Binary (3-Bit)
0	0	000
1	1	001
2	2	010
...	...	...
7	7	111

ÖNEMLİ: Octal sistemin 7'den büyük bir rakamı (8 veya 9) barındırması imkansızdır, çünkü 3-bitlik maksimum binary değer 111 'dir ve bu da 7'ye eşittir.

Octal'dan Decimal'e Çevrim:

Taban aritmetiği değişmez; bu kez tabanımız 8'dir. Örneğin **357** Octal değerini ele alalım:

$$\begin{aligned} 357 \text{ (Octal)} &= (3 \times 8^2) + (5 \times 8^1) + (7 \times 8^0) \\ 357 &= (3 \times 64) + (5 \times 8) + (7 \times 1) \\ 357 &= 192 + 40 + 7 = 239 \text{ (Decimal)} \end{aligned}$$

Bu işlem sonucunda sistemin 357 octal verisini arka planda matematiksel olarak 239 decimal değerine nasıl çevirdiğini mantıksal zincir içerisinde görüyoruz.

Sayı Sistemleri ve Renk Temsili: Nihai Sistem Mimarisi Özeti

Bilgisayar bellek (memory) mimarisinde ve veri işleme süreçlerinde sayısal değerlerin nasıl depolandığını kavramak, siber güvenlikte zafiyet analizi, zararlı yazılım analizi (Malware Analysis) ve tersine mühendislik (Reverse Engineering) süreçlerinin temel taşıdır. İnsan algısı günlük hayattaki ölçümler için **Decimal (Onluk - Base-10)** sistemini kullanacak şekilde evrilmişken, bilgisayar donanımı fiziksel (transistör) seviyesinde sadece iki elektriksel veya manyetik durumu algılayabilir. Bu fiziksel gerçeklik, dijital sistemlerin en alt seviyede sadece 0 ve 1 durumlarından oluşan **Binary (İkili - Base-2)** yapıda çalışmasını mutlak suretle zorunlu kılar.

Bu iki farklı dünya arasındaki veri dönüşümünü yönetmek ve devasa binary yığınlarını insan tarafından okunabilir, yönetilebilir bir formata sokmak için optimizasyon standartları geliştirilmiştir:

Plaintext

Hexadecimal (Base-16) -> 4 bit = 1 Karakter (0-F)
Octal (Base-8) -> 3 bit = 1 Karakter (0-7)

Yukarıdaki yapı, ikili sistemdeki ham (raw) verinin nasıl sıkıştırıldığını gösterir. Bellek adreslerini veya ağ paketlerini incelerken kullanılan Hexadecimal sistem her 4 biti (nibble) tek bir harfe/rakama sığdırır. Daha spesifik alanlarda, özellikle UNIX/Linux dosya izin yapısında (File Permissions) karşımıza çıkan Octal sistem ise veriyi 3 bitlik gruplar halinde işleyerek maksimum 7 değerini barındırır.

Veri Ölçü Birimleri ve 24-Bit Renk (True Color) Derinliği

Sayı sistemlerinin arka plandaki bu akışı, sistemin depolayabileceği en küçük veri parçasından en karmaşık görsel render işlemlerine kadar her şeyi belirler.

Sistem hafızasındaki veri boyutlandırmasının mutlak kuralı şudur:

- **Bit (Binary Digit):** Bir sistemin okuyup yazabileceği en küçük, parçalanamaz veri durumudur (Sadece 0 veya 1).
- **Byte (Octet):** Modern işlemci mimarilerinde **8 adet bit**'in yan yana gelmesiyle oluşan ve hafızada bağımsız olarak adreslenebilen (erişilebilen) en küçük standart veri bloğudur.

Bu donanım kurallarını görsel bir çıktı olan **Hex Color** (Onaltılık Renk) sistemine uyguladığımızda veri boyutunun nasıl ölçeklendiğini net bir şekilde görürüz. Bilgisayar sistemleri herhangi bir rengi ekrana yansıtmak için kırmızı, yeşil ve mavi (RGB) kanallarını kullanır.

ÖNEMLİ: Kusursuz bir görsel derinlik elde etmek adına, bu üç ana rengin her birine tam **1 Byte (8 bit)** alan tahsis edilir. Her bir Byte 256 farklı ışık şiddeti değerini barındırabildiği için, üç kanalın matematiksel çarpımı arka planda devasa bir renk uzayı yaratır. Toplamda **3 Byte (24 bit)** tahsis edilen bu mimari, sistemin 16 milyonun üzerinde ($256 \times 256 \times 256$) benzersiz renk kombinasyonunu kayıpsız olarak işlemlerini ve hafızada saklamasını sağlar.